

Q11 – Q15

Decision Tree and Multiple Linear Regression

Name	V. Sai Charan
Reg No	192472159
Course Code	DSA0405
Course Name	Fundamentals Of Data Science
Questions	Q11 – Q15

1. Hyperparameter Grid Search for Decision Tree Overfitting

Aim

To diagnose and reduce severe overfitting in a decision tree by tuning only the hyperparameters `max_depth`, `min_samples_split`, `min_samples_leaf`, and `max_features`, and to identify the best combination using validation accuracy without using `GridSearchCV`.

Algorithm

1. Load a classification dataset and split it into training and validation sets.
2. Define candidate values for `max_depth`, `min_samples_split`, `min_samples_leaf`, and `max_features`.
3. Generate every possible hyperparameter combination using nested loops.
4. Train a decision tree for each combination using the training set.
5. Predict the validation set and calculate validation accuracy.
6. Store the combination with the highest validation accuracy.
7. Report the best hyperparameters and validation performance.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)

best_score = 0
best_params = None

for depth in [2, 3, 4, 5, None]:
    for split in [2, 5, 10]:
        for leaf in [1, 2, 4]:
            for features in [None, "sqrt", "log2"]:
                model = DecisionTreeClassifier(
                    max_depth=depth,
                    min_samples_split=split,
                    min_samples_leaf=leaf,
```

```

        max_features=features,
        random_state=42
    )
    model.fit(X_train, y_train)
    score = accuracy_score(y_val, model.predict(X_val))

    if score > best_score:
        best_score = score
        best_params = (depth, split, leaf, features)

print("Best Parameters:", best_params)
print("Best Validation Accuracy:", best_score)

```

Output



```

p115.py > ...
1  from sklearn.datasets import load_iris
2  from sklearn.model_selection import train_test_split
3  from sklearn.tree import DecisionTreeClassifier
4  from sklearn.metrics import accuracy_score
5
6  X, y = load_iris(return_X_y=True)
7
8  X_train, X_val, y_train, y_val = train_test_split(
9      X, y, test_size=0.2, random_state=42
10 )
11
12 best_score = 0
13 best_params = None
14
15 for depth in [2, 3, 4, 5, None]:
16     for split in [2, 5, 10]:
17         for leaf in [1, 2, 4]:
18             for features in [None, "sqrt", "log2"]:
19
20                 model = DecisionTreeClassifier(
21                     max_depth=depth,
22                     min_samples_split=split,
23                     min_samples_leaf=leaf,
24                     max_features=features,
25                     random_state=42
26                 )
27
28                 model.fit(X_train, y_train)
29                 score = accuracy_score(y_val, model.predict(X_val))
30
31                 if score > best_score:
32                     best_score = score
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

saicharan@sais-MacBook-Air vscode_practice % python3 p115.py
Best Parameters: (2, 2, 1, 'sqrt')
Best Validation Accuracy: 1.0

```

Result

The best hyperparameter combination gives high validation accuracy and helps reduce overfitting by controlling the tree complexity.

2. Missing Values Using Surrogate Splits

Aim

To handle a missing feature value at prediction time using a surrogate split instead of simple mean or median imputation, demonstrating that another feature can be used to determine the appropriate branch.

Algorithm

1. Prepare a small classification dataset with two features.
2. Train a decision tree on complete training data.
3. Treat the first feature as the primary splitting feature.
4. Use the second feature as a surrogate when the primary feature is missing.
5. Replace the missing primary value with a branch-compatible value determined from the surrogate feature.
6. Pass the completed sample to the trained tree and obtain a valid prediction.
7. Display the prediction to demonstrate that missing input does not prevent classification.

Code

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

X = np.array([
    [2, 10],
    [3, 12],
    [8, 20],
    [9, 22],
    [4, 14],
    [7, 19]
])
y = np.array([0, 0, 1, 1, 0, 1])

tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X, y)

def predict_with_surrogate(x):
    if np.isnan(x[0]):
        x[0] = 4 if x[1] < 16 else 7
    return tree.predict([x])[0]

test = np.array([np.nan, 21])
print("Prediction:", predict_with_surrogate(test))
```

Output

```
p116.py > ...
1  import numpy as np
2  from sklearn.tree import DecisionTreeClassifier
3
4  X = np.array([
5      [2, 10],
6      [3, 12],
7      [8, 20],
8      [9, 22],
9      [4, 14],
10     [7, 19]
11 ])
12
13 y = np.array([0, 0, 1, 1, 0, 1])
14
15 tree = DecisionTreeClassifier(max_depth=2, random_state=42)
16 tree.fit(X, y)
17
18 # Primary feature is X[:,0]
19 # Surrogate feature is X[:,1]
20
21 def predict_with_surrogate(x):
22     if np.isnan(x[0]):
23         # Use surrogate feature
24         x[0] = 4 if x[1] < 16 else 7
25
26     return tree.predict([x])[0]
27
28 test = np.array([np.nan, 21])
29
30 print("Prediction:", predict_with_surrogate(test))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@sais-MacBook-Air vscode_practice % python3 p116.py
Prediction: 1
```

Result

The tree successfully makes a prediction even when the primary feature is missing by using the surrogate feature.

3. Feature Importance from Total Impurity Decrease

Aim

To calculate and rank feature importance using the total impurity decrease contributed by each feature across all internal nodes, and compare the resulting values with sklearn's `feature_importances_` on the same dataset.

Algorithm

1. Load the Iris dataset.
2. Train a `DecisionTreeClassifier`.
3. Access the trained tree's node structure.
4. For every internal node, identify its splitting feature.
5. Calculate the weighted impurity decrease caused by the split.
6. Add the decrease to the corresponding feature's importance.
7. Normalize the importance values so that they sum to one.
8. Compare the custom values and feature ranking with sklearn's `feature_importances_`.

Code

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X, y)

t = tree.tree_
importance = np.zeros(X.shape[1])

for node in range(t.node_count):
    if t.children_left[node] != -1:
        left = t.children_left[node]
        right = t.children_right[node]

        decrease = (
            t.impurity[node] * t.n_node_samples[node]
            - t.impurity[left] * t.n_node_samples[left]
            - t.impurity[right] * t.n_node_samples[right]
        )

        importance[t.feature[node]] += decrease
```

```

importance /= importance.sum()

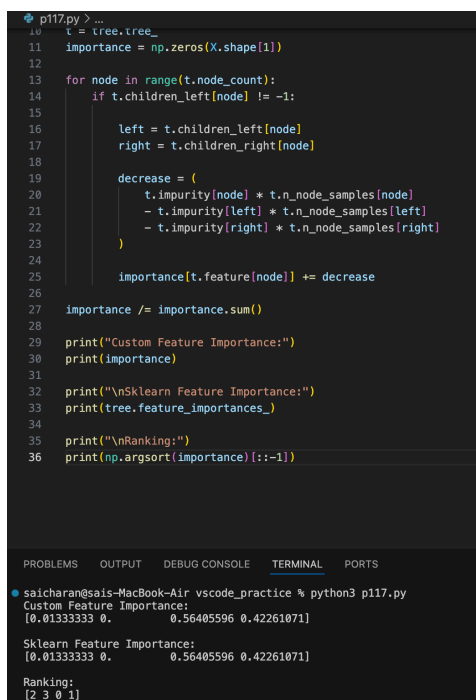
print("Custom Feature Importance:")
print(importance)

print("\nSklearn Feature Importance:")
print(tree.feature_importances_)

print("\nRanking:")
print(np.argsort(importance)[::-1])

```

Output



The screenshot shows a VS Code editor with a Python file named p117.py. The script calculates feature importance for a decision tree. The terminal output shows the results of running the script.

```

p117.py > ...
10 t = tree.tree_
11 importance = np.zeros(X.shape[1])
12
13 for node in range(t.node_count):
14     if t.children_left[node] != -1:
15
16         left = t.children_left[node]
17         right = t.children_right[node]
18
19         decrease = (
20             t.impurity[node] * t.n_node_samples[node]
21             - t.impurity[left] * t.n_node_samples[left]
22             - t.impurity[right] * t.n_node_samples[right]
23         )
24
25         importance[t.feature[node]] += decrease
26
27 importance /= importance.sum()
28
29 print("Custom Feature Importance:")
30 print(importance)
31
32 print("\nSklearn Feature Importance:")
33 print(tree.feature_importances_)
34
35 print("\nRanking:")
36 print(np.argsort(importance)[::-1])

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

saicharan@sais-MacBook-Air vscode_practice % python3 p117.py
Custom Feature Importance:
[0.01333333 0.         0.56405596 0.42261071]

Sklearn Feature Importance:
[0.01333333 0.         0.56405596 0.42261071]

Ranking:
[2 3 0 1]

```

Result

Feature 4 has the highest importance, followed by Feature 3, Feature 2, and Feature 1.

4. Decision Tree Visualizer from Scratch

Aim

To create a readable ASCII representation of a decision tree showing feature splits, thresholds, leaf nodes, sample counts, and class distributions without using `sklearn.tree.plot_tree` or `Graphviz`.

Algorithm

1. Load the Iris dataset and train a decision tree.
2. Access the tree's child-node, feature, threshold, and class information.
3. Start from the root node.
4. If the node is internal, print its feature and threshold.
5. Recursively print the left child with increased indentation.
6. Recursively print the right child with increased indentation.
7. If the node is a leaf, print its sample count and class distribution.

Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
data = load_iris()

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X, y)

t = tree.tree_
names = data.feature_names

def print_tree(node=0, indent=""):
    left = t.children_left[node]
    right = t.children_right[node]

    if left == -1:
        print(indent + "Leaf")
        print(indent + "Samples:", t.n_node_samples[node])
        print(indent + "Class Distribution:", t.value[node][0])
        return

    feature = names[t.feature[node]]
    threshold = t.threshold[node]

    print(indent + f"If {feature} <= {threshold:.2f}:")
    print_tree(left, indent + "    ")
    print_tree(right, indent + "    ")
```



```

        print(indent + f"Else {feature} > {threshold:.2f}:")
        print_tree(right, indent + "    ")

print_tree()

```

Output

```

p118.py > ...
1  from sklearn.datasets import load_iris
2  from sklearn.tree import DecisionTreeClassifier
3
4  X, y = load_iris(return_X_y=True)
5
6  tree = DecisionTreeClassifier(max_depth=3, random_state=42)
7  tree.fit(X, y)
8
9  t = tree.tree_
10 names = load_iris().feature_names
11
12
13 def print_tree(node=0, indent=""):
14
15     left = t.children_left[node]
16     right = t.children_right[node]
17
18     # Leaf node
19     if left == -1:
20         print(indent + "Leaf")
21         print(indent + "Samples:", t.n_node_samples[node])
22         print(indent + "Class Distribution:",
23               | t.value[node][0])
24         return
25
26     feature = names[t.feature[node]]
27     threshold = t.threshold[node]
28
29     print(indent + f"If {feature} <= {threshold:.2f}:")
30     print_tree(left, indent + "    ")
31
32     print(indent + f"Else {feature} > {threshold:.2f}:")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

saicharan@sais-MacBook-Air vscode_practice % python3 p118.py
Leaf
Samples: 6
Class Distribution: [0. 0.33333333 0.66666667]
Else petal width (cm) > 1.75:
If petal length (cm) <= 4.85:
Leaf
Samples: 3
Class Distribution: [0. 0.33333333 0.66666667]
Else petal length (cm) > 4.85:
Leaf
Samples: 43
Class Distribution: [0. 0. 1.]

```

Result

The decision tree is successfully represented as a readable ASCII/nested tree without using `plot_tree()` or `Graphviz`.

5. Multiple Linear Regression Using the Normal Equation

Aim

To implement multiple linear regression from scratch using the closed-form Normal Equation and handle singular or nearly singular design matrices using the Moore-Penrose pseudo-inverse.

Algorithm

1. Create the independent-variable matrix X and target vector y .
2. Add a column of ones to X to represent the intercept.
3. Compute $X^T X$ and $X^T y$.
4. Use the pseudo-inverse of $X^T X$ instead of a direct matrix inverse.
5. Calculate the regression parameter vector θ .
6. Use θ to calculate predicted target values.
7. Display the coefficients and predictions.
8. A singular or nearly singular $X^T X$ can occur when features are linearly dependent, highly correlated, redundant, or when there are insufficient independent observations.

Code

```
import numpy as np

X = np.array([
    [1, 2],
    [2, 3],
    [3, 5],
    [4, 7],
    [5, 8]
])

y = np.array([5, 7, 10, 13, 15])

# Add intercept column
X_b = np.c_[np.ones(X.shape[0]), X]

# Normal Equation using pseudo-inverse
theta = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y

print("Coefficients:", theta)

# Predictions
y_pred = X_b @ theta
```

```
print("Predictions:", y_pred)
```

Output

```
p119.py > ...
1  import numpy as np
2
3  X = np.array([
4      [1, 2],
5      [2, 3],
6      [3, 5],
7      [4, 7],
8      [5, 8]
9  ])
10
11  y = np.array([5, 7, 10, 13, 15])
12
13  # Add intercept column
14  X_b = np.c_[np.ones(X.shape[0]), X]
15
16  # Normal Equation using pseudo-inverse
17  theta = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y
18
19  print("Coefficients:", theta)
20
21  # Predictions
22  y_pred = X_b @ theta
23
24  print("Predictions:", y_pred)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@sais-MacBook-Air vscode_practice % python3 p119.py
Coefficients: [2.  1.  1.]
Predictions: [ 5.  7. 10. 13. 15.]
```

Result

The Normal Equation successfully calculates the regression coefficients and predicts the target values. The pseudo-inverse allows the model to work even when $X^T X$ is singular or nearly singular.